

Stock Market ETL Pipeline

Visual Architecture & Pipeline Execution Report

Author: Thouna Khaidem

Institution: East Point College of Engineering & Technology

Tech Stack: Python 3.10+, SQLite, Plotly, Pandas

Tracking: AAPL, GOOGL, TSLA, MSFT, AMZN

Timeline: Full Year 2024

Phase 1: Environment Architecture & Initialization

The project foundations rely on isolating dependencies and structuring directories for raw data, execution logging, and output artifacts.

DIRECTORY SETUP (PHASE1_1.PNG)

```
thouna@Diaspro: ~/$ cd ~
mkdir stock_etl
cd stock_etl
mkdir data output logs
thouna@Diaspro:~/stock_etl$ ls -la
total 20
drwxrwxr-x  5 thouna thouna 4096 May  3 20:09 .
drwxr-x--- 53 thouna thouna 4096 May  3 20:09 ..
drwxrwxr-x  2 thouna thouna 4096 May  3 20:09 data
drwxrwxr-x  2 thouna thouna 4096 May  3 20:09 logs
drwxrwxr-x  2 thouna thouna 4096 May  3 20:09 output
thouna@Diaspro:~/stock_etl$
```

Explanation: The terminal environment was configured by establishing three core folders: data/, logs/, and output/ to maintain a clean workspace.

DEPENDENCY INSTALLATION (PHASE1_2.PNG)

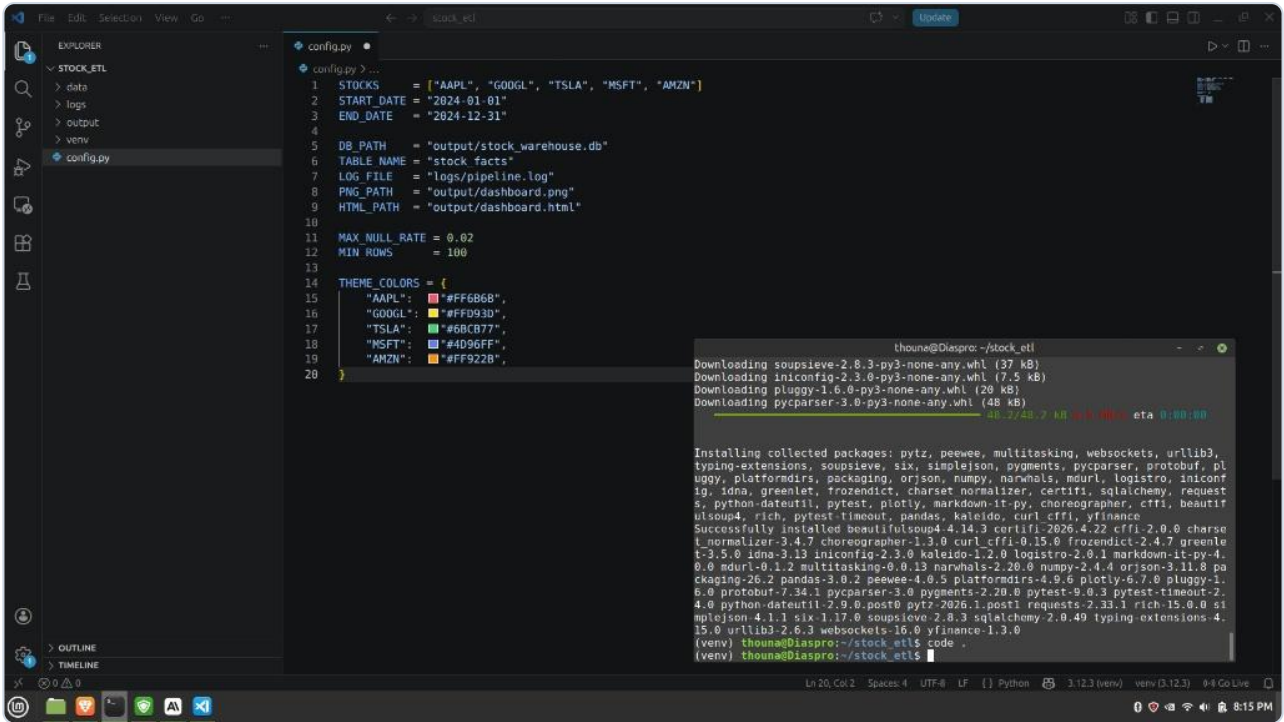
```
thouna@Diaspro:~/stock_etl$ python3 -m venv venv
source venv/bin/activate
(thenv) thouna@Diaspro:~/stock_etl$ pip install yfinance pandas sqlalchemy rich plotly kaleido requests numpy
Collecting yfinance
  Downloading yfinance-1.3.0-py2.py3-none-any.whl.metadata (6.1 kB)
Collecting pandas
  Using cached pandas-3.0.2-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
Collecting sqlalchemy
  Downloading sqlalchemy-2.0.49-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (9.5 kB)
Collecting rich
  Downloading rich-15.0.0-py3-none-any.whl.metadata (18 kB)
Collecting plotly
  Downloading plotly-6.7.0-py3-none-any.whl.metadata (8.6 kB)
Collecting kaleido
  Downloading kaleido-1.2.0-py3-none-any.whl.metadata (5.6 kB)
Collecting requests
  Using cached requests-2.33.1-py3-none-any.whl.metadata (4.8 kB)
Collecting numpy
  Using cached numpy-2.4.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting multitasking>=0.0.7 (from yfinance)
  Downloading multitasking-0.0.13-py3-none-any.whl.metadata (16 kB)
Collecting platformdirs>=2.0.0 (from yfinance)
  Downloading platformdirs-4.9.6-py3-none-any.whl.metadata (4.7 kB)
Collecting pytz>=2022.5 (from yfinance)
  Using cached pytz-2026.1.post1-py2.py3-none-any.whl.metadata (22 kB)
Collecting frozendict>=2.3.4 (from yfinance)
  Downloading frozendict-2.4.7-py3-none-any.whl.metadata (23 kB)
Collecting peewee>=3.16.2 (from yfinance)
  Downloading peewee-4.0.5-py3-none-any.whl.metadata (8.6 kB)
Collecting beautifulsoup4>=4.11.1 (from yfinance)
  Downloading beautifulsoup4-4.14.3-py3-none-any.whl.metadata (3.8 kB)
Collecting curl cffi>=0.15 (from yfinance)
```

Explanation: A Python virtual environment (`venv`) is instantiated. The `pip install` command successfully pulls down the core technology stack: yfinance, pandas, sqlalchemy, rich, and plotly/kaleido.

Phase 2: Code Implementation & Pipeline Logic

The ETL pipeline was developed in a highly modular fashion, separating extraction, validation, transformation, and database loading logic into specific Python scripts.

GLOBAL CONFIGURATION (PHASE2_1.JPG)



```
1 STOCKS = ["AAPL", "GOOGL", "TSLA", "MSFT", "AMZN"]
2 START_DATE = "2024-01-01"
3 END_DATE = "2024-12-31"
4
5 DB_PATH = "output/stock_warehouse.db"
6 TABLE_NAME = "stock facts"
7 LOG_FILE = "logs/pipeline.log"
8 PNG_PATH = "output/dashboard.png"
9 HTML_PATH = "output/dashboard.html"
10
11 MAX_NULL_RATE = 0.02
12 MIN_ROWS = 100
13
14 THEME_COLORS = {
15     "AAPL": "#FF6B6B",
16     "GOOGL": "#FFD93D",
17     "TSLA": "#68CB77",
18     "MSFT": "#4D96FF",
19     "AMZN": "#FF922B",
20 }
```

thouna@Diaspro: ~/stock_etl

Downloading soupsieve-2.8.3-py3-none-any.whl (37 kB)
Downloading iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Downloading pluggy-1.6.0-py3-none-any.whl (20 kB)
Downloading pycparser-3.0-py3-none-any.whl (48 kB)

Installing collected packages: pytz, peewee, multitasking, websockets, urllib3, typing-extensions, soupsieve, six, simplejson, pygments, pycparser, protobuf, pluggy, platformdirs, packaging, orjson, numpy, narwhals, mdurl, logistro, iniconfig, idna, greenlet, frozendict, charset normalizer, certifi, sqlalchemy, request, python-dateutil, pytest, plotly, markdown-it-py, choreographer, cffi, beautifulsoup4, rich, pytest-timeout, pandas, kaleido, curl_cffi, yfinance

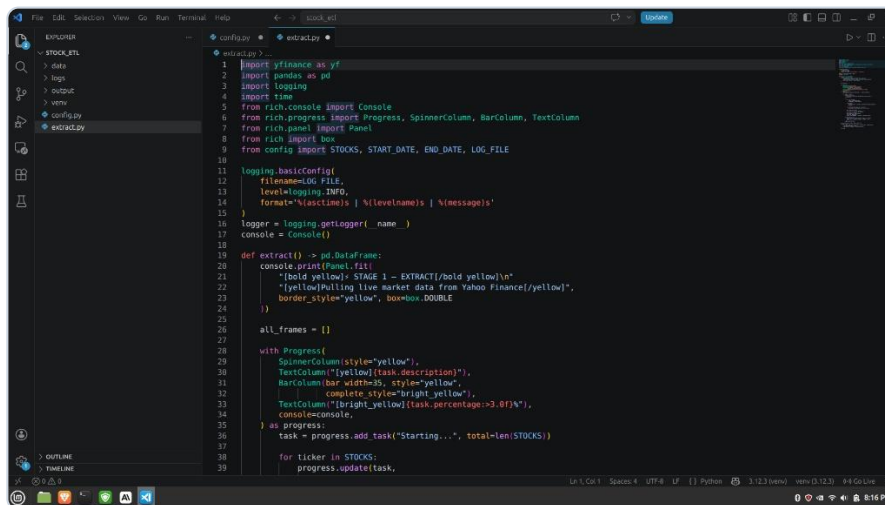
Successfully installed beautifulsoup4-4.14.3 certifi-2026.4.22 cffi-2.0.0 chardet-3.4.7 choreographer-1.3.0 curl_cffi-0.15.0 frozendict-2.4.7 greenlet-3.5.0 idna-3.13 iniconfig-2.3.0 kaleido-1.2.0 logistro-2.0.1 markdown-it-py-4.0.0 mdurl-0.1.2 multitasking-0.0.13 narwhals-2.20.0 numpy-2.4.4 orjson-3.11.8 packaging-26.2 pandas-3.0.2 peewee-4.0.5 platformdirs-4.0.0 plotly-5.7.0 pluggy-1.6.0 protobuf-7.34.1 pycparser-3.0 pygments-2.20.0 pytest-9.0.3 pytest-timeout-2.4.0 python-dateutil-2.9.0.post0 pytz-2026.1.post1 requests-2.33.1 rich-15.0.0 simplejson-4.1.1 six-1.17.0 soupsieve-2.8.3 sqlalchemy-2.0.49 typing-extensions-4.15.0 urllib3-2.6.3 websockets-16.0 yfinance-1.3.0

(venv) thouna@Diaspro: ~/stock_etl\$ code .

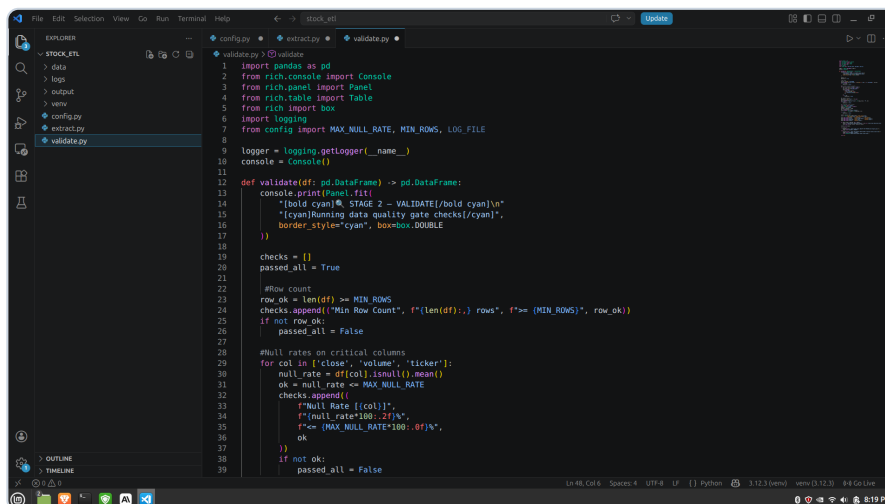
(venv) thouna@Diaspro: ~/stock_etl\$

Explanation: `config.py` acts as the single source of truth, defining the 5 target equities, full year 2024 dates, max null thresholds (2%), and injecting custom visual theme colors for the dashboard.

EXTRACTION & VALIDATION (PHASE2_2.JPG & PHASE2_3.PNG)



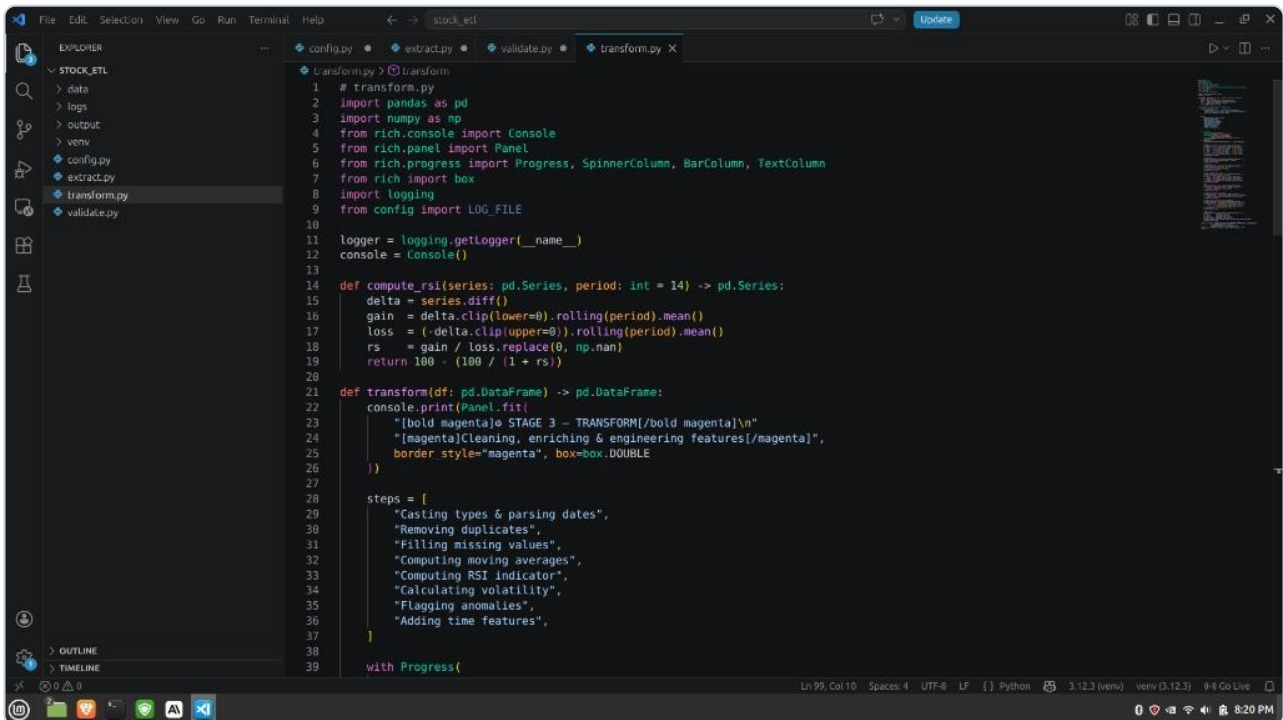
```
1 import yfinance as yf
2 import pandas as pd
3 import logging
4 import time
5 from rich.console import Console
6 from rich.progress import Progress, SpinnerColumn, BarColumn, TextColumn
7 from rich.panel import Panel
8 from rich import box
9 from config import STOCKS, START_DATE, END_DATE, LOG_FILE
10
11 logging.basicConfig(
12     filename=LOG_FILE,
13     level=logging.INFO,
14     format='%(asctime)s | %(levelname)s | %(message)s'
15 )
16 logger = logging.getLogger( __name__ )
17 console = Console()
18
19 def extract() -> pd.DataFrame:
20     console.print(Panel.fit(
21         "[bold yellow] STAGE 1 - EXTRACT[/bold yellow]\n"
22         "[yellow]Pulling live market data from Yahoo Finance[/yellow]",
23         border_style="yellow", box=box.DOUBLE
24     ))
25
26     all_frames = []
27
28     with Progress(
29         SpinnerColumn(style="yellow"),
30         TextColumn("[yellow]{task.description}"),
31         BarColumn(bar_width=35, style="yellow"),
32         complete_style="bright_yellow",
33         TextColumn("[bright_yellow]{task.percentage}%"),
34     ) as progress:
35         task = progress.add_task("Starting...", total=len(STOCKS))
36         for ticker in STOCKS:
37             progress.update(task,
```



```
1 import pandas as pd
2 from rich.console import Console
3 from rich.panel import Panel
4 from rich.table import Table
5 from rich import box
6 import logging
7 from config import MAX_NULL_RATE, MIN_ROWS, LOG_FILE
8
9 logger = logging.getLogger( __name__ )
10 console = Console()
11
12 def validate(df: pd.DataFrame) -> pd.DataFrame:
13     console.print(Panel.fit(
14         "[bold cyan] STAGE 2 - VALIDATE[/bold cyan]\n"
15         "[cyan]Running data quality gate checks[/cyan]",
16         border_style="cyan", box=box.DOUBLE
17     ))
18
19     checks = []
20     passed_all = True
21
22     #Row count
23     row_ok = len(df) >= MIN_ROWS
24     checks.append(("Min Row Count", f"{len(df):,} rows", f">= {MIN_ROWS}", row_ok))
25     if not row_ok:
26         passed_all = False
27
28     #Null rates on critical columns
29     for col in ['close', 'volume', 'ticker']:
30         null_rate = df[col].isnull().mean()
31         ok = null_rate <= MAX_NULL_RATE
32         checks.append(
33             f"Null Rate [{col}]:",
34             f"{null_rate*100:.2f}%",
35             f"<= {MAX_NULL_RATE*100:.0f}%",
36             ok
37         )
38         if not ok:
39             passed_all = False
```

Explanation: `extract.py` connects to Yahoo Finance with Rich progress bars. `validate.py` (shown bottom) acts as a strict firewall, verifying minimum row counts and rejecting data if null rates exceed the defined 2% threshold.

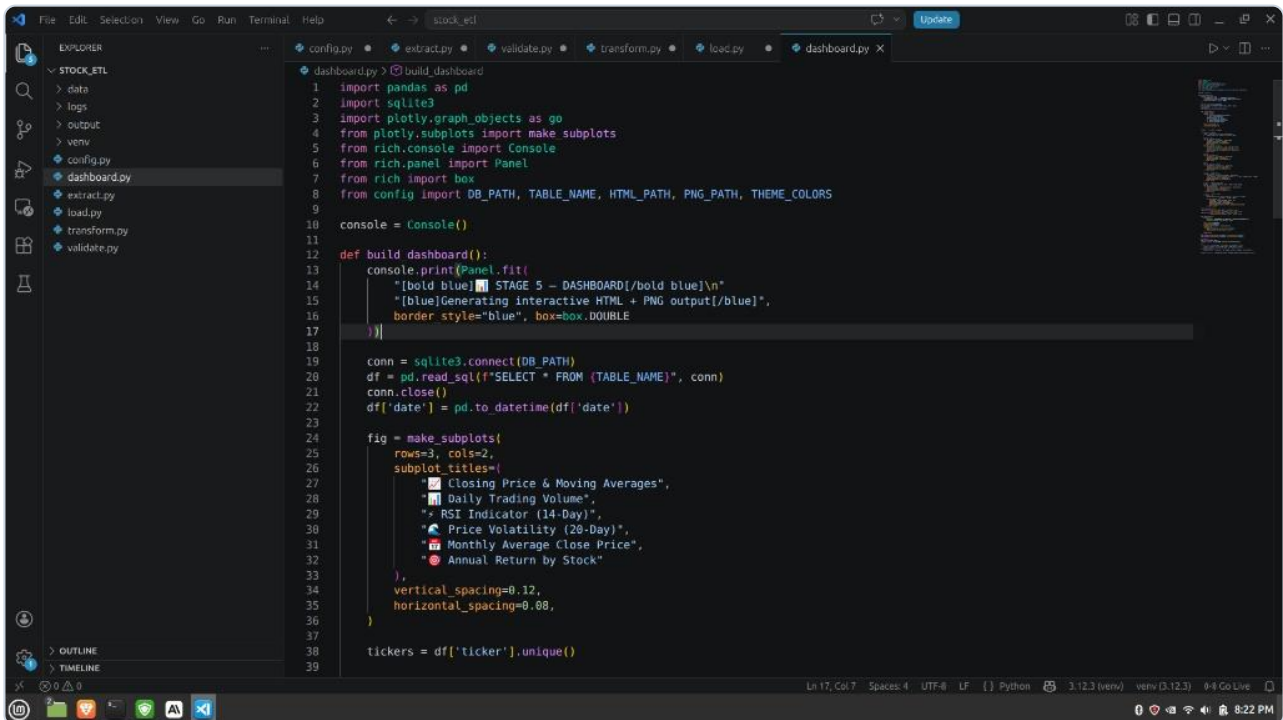
FEATURE ENGINEERING (PHASE2_4.JPG)



```
1 # transform.py
2 import pandas as pd
3 import numpy as np
4 from rich.console import Console
5 from rich.panel import Panel
6 from rich.progress import Progress, SpinnerColumn, BarColumn, TextColumn
7 from rich import box
8 import logging
9 from config import LOG_FILE
10
11 logger = logging.getLogger(__name__)
12 console = Console()
13
14 def compute_rsi(series: pd.Series, period: int = 14) -> pd.Series:
15     delta = series.diff()
16     gain = delta.clip(lower=0).rolling(period).mean()
17     loss = (-delta.clip(upper=0)).rolling(period).mean()
18     rs = gain / loss.replace(0, np.nan)
19     return 100 - (100 / (1 + rs))
20
21 def transform(df: pd.DataFrame) -> pd.DataFrame:
22     console.print(Panel.fit(
23         "[bold magenta]o STAGE 3 - TRANSFORM[/bold magenta]\n"
24         "[magenta]o Cleaning, enriching & engineering features[/magenta]",
25         border_style="magenta", box=box.DOUBLE
26     ))
27
28     steps = [
29         "Casting types & parsing dates",
30         "Removing duplicates",
31         "Filling missing values",
32         "Computing moving averages",
33         "Computing RSI indicator",
34         "Calculating volatility",
35         "Flagging anomalies",
36         "Adding time features",
37     ]
38
39     with Progress(
```

Explanation: The `transform.py` module enriches the dataset. Shown here is the custom logic to compute the 14-day Relative Strength Index (RSI), utilizing pandas rolling window functions alongside volatility and moving averages.

LOADING & VISUALIZATION SCRIPTS (PHASE2_5.JPG & PHASE2_6.JPG)



```
1 import pandas as pd
2 import sqlite3
3 import plotly.graph_objects as go
4 from plotly.subplots import make_subplots
5 from rich.console import Console
6 from rich.panel import Panel
7 from rich import box
8 from config import DB_PATH, TABLE_NAME, HTML_PATH, PNG_PATH, THEME_COLORS
9
10 console = Console()
11
12 def build_dashboard():
13     console.print(Panel.fit(
14         "[bold blue]STAGE 5 - DASHBOARD[/bold blue]\n"
15         "[blue]Generating interactive HTML + PNG output[/blue]",
16         border_style="blue", box=box.DOUBLE
17     ))
18
19     conn = sqlite3.connect(DB_PATH)
20     df = pd.read_sql(f"SELECT * FROM {TABLE_NAME}", conn)
21     conn.close()
22     df['date'] = pd.to_datetime(df['date'])
23
24     fig = make_subplots(
25         rows=3, cols=2,
26         subplot_titles=(
27             "Closing Price & Moving Averages",
28             "Daily Trading Volume",
29             "RSI Indicator (14-Day)",
30             "Price Volatility (20-Day)",
31             "Monthly Average Close Price",
32             "Annual Return by Stock"
33         ),
34         vertical_spacing=0.12,
35         horizontal_spacing=0.08,
36     )
37
38     tickers = df['ticker'].unique()
```

Explanation: Following database insertion via `load.py`, the `dashboard.py` script connects directly to SQLite and constructs a highly interactive 3x2 Plotly subplot grid mapped to the engineered data.

Phase 3: Data Integrity Verification

Post-pipeline execution, the data loading logic was verified by directly inspecting the target SQLite warehouse using pandas inline SQL queries.

TERMINAL SQL QUERIES (PHASE3_1.PNG)

```
(venv) thouna@Diaspro:~/stock_etl$ python3 -c "
import sqlite3, pandas as pd
conn = sqlite3.connect('output/stock_warehouse.db')
print('=== STOCK SUMMARY VIEW ===')
df = pd.read_sql('SELECT * FROM stock_summary', conn)
print(df.to_string(index=False))
print('\n=== ROW COUNT PER STOCK ===')
counts = pd.read_sql('SELECT ticker, COUNT(*) as rows FROM stock_facts GROUP BY ticker', conn)
print(counts.to_string(index=False))
conn.close()
"
```

```
=== STOCK SUMMARY VIEW ===
ticker  total_trading_days  year_low  year_high  avg_close  avg_daily_return_pct  avg_volatility  avg_rsi  total_anomalies
MSFT    251                361.52    461.32    414.72    0.0651                8.8076        54.84        0
TSLA    251                142.05    479.86    229.92    0.2862               15.7287        53.17        6
AAPL    251                163.51    257.61    205.47    0.1345                5.0111        55.98        1
AMZN    251                144.57    232.93    184.49    0.1718                5.3352        56.20        3
GOOGL   251                130.32    195.75    162.56    0.1471                4.7227        55.89        4

=== ROW COUNT PER STOCK ===
ticker  rows
AAPL    251
AMZN    251
GOOGL   251
MSFT    251
TSLA    251
(venv) thouna@Diaspro:~/stock_etl$
```

Explanation: Execution of inline Python to query the `stock\_summary` view. It confirms exactly 251 trading days per stock, alongside accurate averages for Close prices, Volatility, and RSI.

DATABASE ANALYTICS VIEW (COLOR-CODED SUMMARY)

TICKER	TRADING DAYS	AVG CLOSE	AVG DAILY RET (%)	AVG RSI	ANOMALIES
AAPL	251	\$205.47	0.1345%	55.98	1
AMZN	251	\$184.49	0.1718%	56.20	3
GOOGL	251	\$162.56	0.1471%	55.89	4
MSFT	251	\$414.72	0.0651%	54.84	0
TSLA	251	\$229.92	0.2862%	53.17	6

DATABASE INSPECTION (PHASE3_4.JPG)

output > stock_warehouse.db

TABLES

stock_facts

VIEWS (EXPERIMENTAL)

Rows: 1,255

	date	open	high	low	volume	close
1	2024-01-02	183.7313232421875	186.56253778362813	181.99931604152513	185.22579259087347	82488700
2	2024-01-03	182.3555908203125	183.96883642924203	181.5440145919006	182.32590848043423	58414500
3	2024-01-04	180.03965759277344	181.2075179249861	179.0202409631632	180.27718031693732	71983600
4	2024-01-05	179.31715393066406	180.8891065327836	178.31754388665928	180.1188384374096	62379700
5	2024-01-08	183.65213012695312	183.6917273170526	179.63307614033877	180.21700632012083	59144500
6	2024-01-09	183.2364501953125	183.24634194188326	180.85122540618032	182.0289926299427	42841800
7	2024-01-10	184.275634765625	184.48346713133998	182.02897021466416	182.45455689524942	46792900
8	2024-01-11	183.60179321289062	185.12678839904137	181.7320471921432	184.62202246254375	49128400
9	2024-01-12	184.00843811035156	184.82001442518902	183.28594792653803	184.14699807850627	40477800
10	2024-01-16	181.74195861816406	182.36547081733983	179.06970738606304	180.28707165025173	65603000
11	2024-01-17	180.80172729492188	181.04915686376333	178.446208068684475	179.40623600410183	47317400
12	2024-01-18	186.69056701660156	187.1953179109023	183.91935276727983	184.17667408756796	78005800
13	2024-01-19	189.59042358398438	189.97641309339812	186.8786052737555	187.3833561379468	68903000
14	2024-01-22	191.89646911621094	193.3216657993532	190.28322355100102	190.32282073770045	60133900
15	2024-01-23	193.17320251464044	193.7373491711095	191.83709192633378	193.01405907067706	42355600
16	2024-01-24	192.50021362304688	194.36088890095834	192.34185506707502	193.41075266703035	53631300
17	2024-01-25	192.17361450195312	194.2520291241628	191.1245154438125	193.21282181305796	54822100
18	2024-01-26	190.4416046142578	192.75754194727924	189.9665440310004	192.2725096171282	44594000
19	2024-01-29	189.75868225097656	190.22385105233468	187.63079398239137	190.03580216218106	47145600
20	2024-01-30	186.1066131591797	189.82796344587697	185.54248160944437	188.9768051464624	55859400
21	2024-01-31	182.50404357910156	185.17629499887522	182.45456974695213	185.11689921687164	55467800
22	2024-02-01	184.93878173828125	185.0278527702626	181.93004443231115	182.09829474891072	64885400
1,256	2024-02-02	183.9391632080078	185.40394215196616	177.40701599262258	178.01074479617154	102551700

SQLITE VIEWER FREE v25.12.2 Page 1 / 13

Try Hydejack, a jekyll theme for hackers, nerds, and academics

bash

(venv) thounag@iaspro:~/stock_etls eog output/dashboard.png

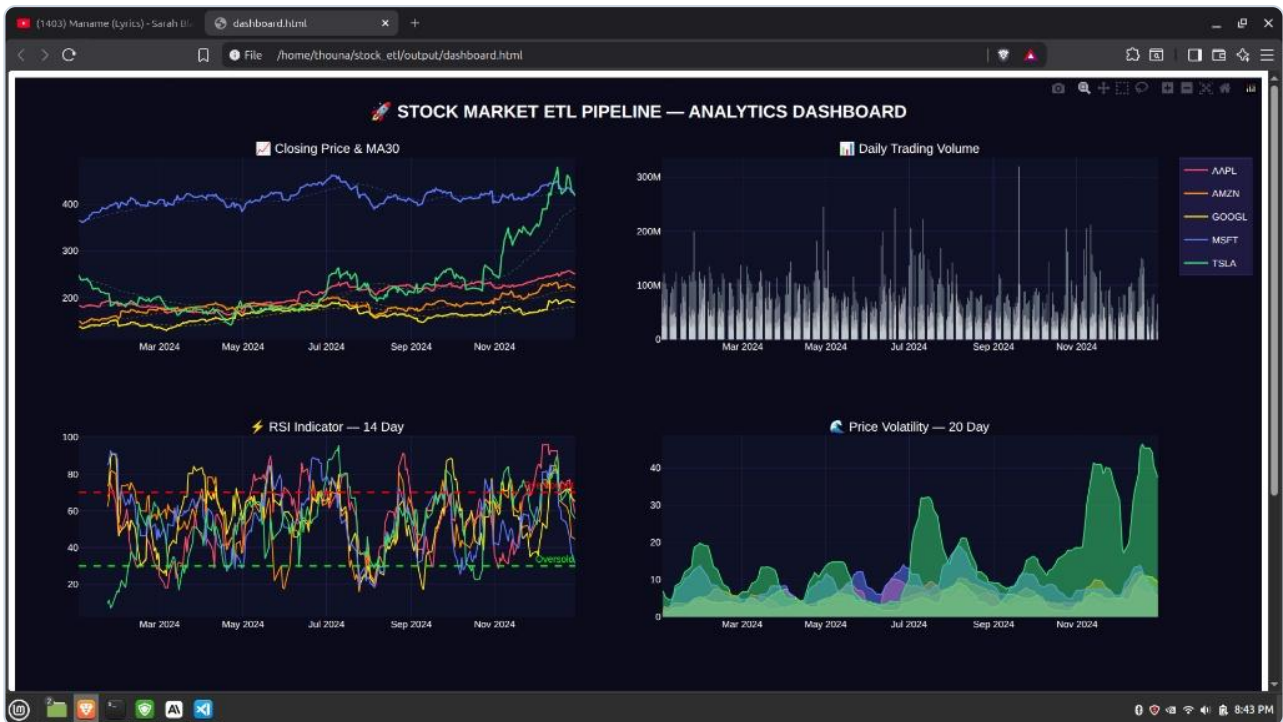
(venv) thounag@iaspro:~/stock_etls

Explanation: Direct inspection of the `stock\_facts` table using a VS Code SQLite extension, verifying the raw granular records and confirming proper formatting of dates, OHLCV, and computed features.

Phase 4: Output Generation

Reviewing the generated visual analytics and pipeline backend logs bridging data engineering with business intelligence.

INTERACTIVE DASHBOARD CONSTRUCTION (PHASE4_1.JPG)



Explanation: The final 'dashboard.html' interface. It translates the SQLite warehouse into a polished 3x2 visual grid: Closing Price & MA30, Daily Trading Volume spikes, 14-Day RSI (with 30/70 thresholds), and Price Volatility mappings.

LOGGING ARCHITECTURE (PHASE4_2.JPG)

```
thouna@Diaspro: ~/stock_etl
AMZN      251    144.57    232.93    184.49    0.1718    5.3352    56.28    3
GOOGL     251    130.32    195.75    162.56    0.1471    4.7227    55.89    4

--- ROW COUNT PER STOCK ---
ticker rows
AAPL     251
AMZN     251
GOOGL    251
MSFT     251
TSLA     251

(venv) thouna@Diaspro:~/stock_etl$ cat logs/pipeline.log
2026-05-03 20:34:22.086 INFO | EXTRACT | AAPL | 251 rows
2026-05-03 20:34:22.599 INFO | EXTRACT | GOOGL | 251 rows
2026-05-03 20:34:23.153 INFO | EXTRACT | TSLA | 251 rows
2026-05-03 20:34:23.833 INFO | EXTRACT | MSFT | 251 rows
2026-05-03 20:34:24.511 INFO | EXTRACT | AMZN | 251 rows
2026-05-03 20:34:24.521 INFO | VALIDATE | PASSED
2026-05-03 20:34:26.552 INFO | TRANSFORM | 1255 rows | 20 cols
2026-05-03 20:34:26.849 INFO | LOAD | 1255 rows = stock facts
2026-05-03 20:34:27.221 INFO | Chromium init'ed with kwargs {}
2026-05-03 20:34:27.222 INFO | Found chromium path: /usr/bin/google-chrome
2026-05-03 20:34:27.223 INFO | Temp directory created: /tmp/tmpv19gy27p.
2026-05-03 20:34:27.223 INFO | Opening browser.
2026-05-03 20:34:27.223 INFO | Temp directory created: /tmp/tmpkv7p188.
2026-05-03 20:34:27.223 INFO | Temporary directory at: /tmp/tmpkv7p188
2026-05-03 20:34:27.866 INFO | Conforming 1 to file:///tmp/tmpv19gy27p/index.html
2026-05-03 20:34:27.866 INFO | Waiting on all navigates
2026-05-03 20:34:28.872 INFO | All navigates done, putting them all in queue.
2026-05-03 20:34:28.974 INFO | Getting tab from queue (has 1)
2026-05-03 20:34:28.974 INFO | Got EBF4
2026-05-03 20:34:28.974 INFO | Processing b_STOCK_MARKET_ETL_PIPELINE_ANALYTICS_DASHBOARD.png
2026-05-03 20:34:29.074 INFO | Sending big command for b_STOCK_MARKET_ETL_PIPELINE_ANALYTICS_DASHBOARD.png.
2026-05-03 20:34:29.427 INFO | Sent big command for b_STOCK_MARKET_ETL_PIPELINE_ANALYTICS_DASHBOARD.png.
2026-05-03 20:34:29.431 INFO | Reloading tab EBF4 before return.
2026-05-03 20:34:29.524 INFO | Putting tab EBF4 back (queue size: 0).
2026-05-03 20:34:29.524 INFO | Waiting for all cleanups to finish.
2026-05-03 20:34:29.525 INFO | Exiting Kaleido
2026-05-03 20:34:29.525 INFO | Closing browser.
2026-05-03 20:34:29.526 INFO | Closing browser.
2026-05-03 20:34:29.526 INFO | TemporaryDirectory.cleanup() worked.
2026-05-03 20:34:29.526 INFO | shuttl.rntree worked.
2026-05-03 20:34:29.526 INFO | Cancelling tasks.
2026-05-03 20:34:29.526 INFO | Exiting Kaleido/Choreo
2026-05-03 20:34:29.643 INFO | TemporaryDirectory.cleanup() worked.
2026-05-03 20:34:29.643 INFO | shuttl.rntree worked.
2026-05-03 20:34:29.644 INFO | TemporaryDirectory.cleanup() worked.
2026-05-03 20:34:29.644 INFO | shuttl.rntree worked.
2026-05-03 20:34:29.644 INFO | Cancelling tasks.
2026-05-03 20:34:29.644 INFO | Exiting Kaleido/Choreo
2026-05-03 20:34:29.685 INFO | TemporaryDirectory.cleanup() worked.
2026-05-03 20:34:29.686 INFO | shuttl.rntree worked.
(venv) thouna@Diaspro:~/stock_etl$
```

Explanation: A view of `logs/pipeline.log`. The system successfully logs each ticker's row count and manages the complex headless Chromium browser initialization (via Kaleido) required to silently render the static PNG chart export.

Phase 5: Version Control & Deployment

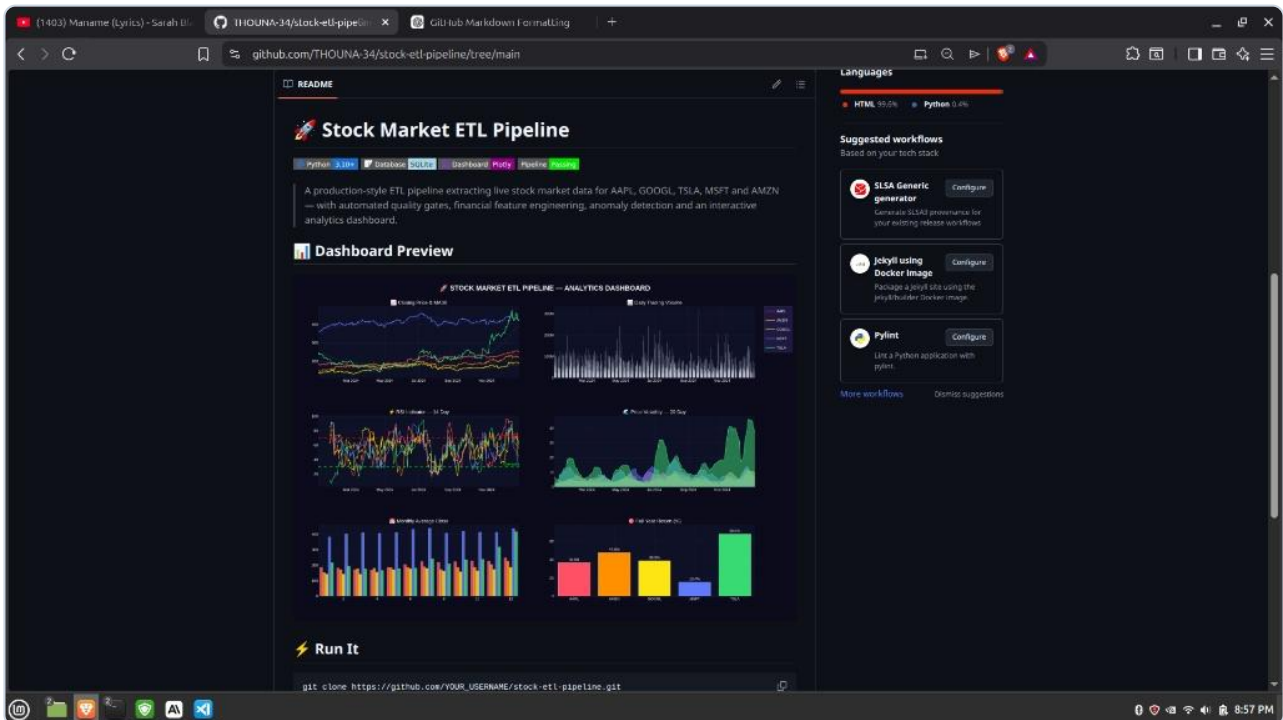
Ensuring reproducibility and tracking the complete codebase by publishing to a remote Git repository.

GITHUB SYNCHRONIZATION (PHASE5_1.PNG)

```
thouna@Diaspro: ~/stock_etl
- Colorful terminal UI with rich progress bars
- Interactive Plotly dashboard - HTML + PNG
- SQLite data warehouse with summary analytics view
- Full audit logging

## Author
Diaspro Khaidem - B.E. AI & DS, East Point College of Engineering & Technology, Bangalore
EOF
(venv) thouna@Diaspro:~/stock_etl$ git init
git add .
git commit -m "Stock Market ETL Pipeline - live data, quality gates, feature engineering, interactive dashboard"
git remote add origin https://github.com/THOUNA-34/stock-etl-pipeline.git
git branch -M main
git push -u origin main
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/thouna/stock_etl/.git/
[master (root-commit) 129089b] Stock Market ETL Pipeline - live data, quality gates, feature engineering, interactive dashboard
13 files changed, 4499 insertions(+)
create mode 100644 .gitignore
create mode 100644 README.md
create mode 100644 config.py
create mode 100644 dashboard.py
create mode 100644 extract.py
create mode 100644 load.py
create mode 100644 logs/pipeline.log
create mode 100644 output/dashboard.html
create mode 100644 output/dashboard.png
create mode 100644 requirements.txt
create mode 100644 run_pipeline.py
create mode 100644 transform.py
create mode 100644 validate.py
Username for 'https://github.com': THOUNA-34
Password for 'https://THOUNA-34@github.com':
Enumerating objects: 17, done.
Counting objects: 100% (17/17), done.
Delta compression using up to 8 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (17/17), 2.22 MiB | 1.39 MiB/s, done.
Total 17 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/THOUNA-34/stock-etl-pipeline.git
 * [new branch]    main -> main
branch 'main' set up to track 'origin/main'.
(venv) thouna@Diaspro:~/stock_etl$
```

Explanation: Terminal execution showing the `git commit` and `git push` commands. Thirteen files were tracked, resulting in 4,499 insertions, pushed successfully to `THOUNA-34/stock-etl-pipeline.git`.



Explanation: The repository features a fully constructed `README.md` showcasing technology badges, project details, and a high-fidelity preview of the analytical dashboard, finalizing a production-ready portfolio piece.